João Yutaka Kato de Oliveira; Lucas Philippe Nunes de Lima; Pedro Henrique Pereira

Educação Gamificada: Um Jogo para o Ensino de Lógica de Programação

Trabalho de Conclusão de Curso apresentado ao curso de Bacharelado em Engenharia de Software da Pontifícia Universidade Católica do Paraná como requisito parcial para obtenção do título Bacharel em Engenharia de Software.

Área de concentração: Engenharia de Software

Orientador: Tiago Adelino Navarro

Curitiba
2025

RESUMO

Este Trabalho de Conclusão de Curso propõe o desenvolvimento de um jogo digital educacional voltado ao ensino de lógica de programação para estudantes do ensino superior. A proposta busca mitigar as dificuldades enfrentadas por alunos iniciantes, como a abstração de conceitos, raciocínio lógico e sintaxe, por meio de uma abordagem lúdica e interativa. O jogo incorpora elementos da técnica Programming by Demonstration (PBD), aprendizagem tangencial e gamificação, permitindo que o jogador explore os conceitos fundamentais da lógica de programação sem a necessidade de codificação textual. A proposta foca em apresentar uma alternativa eficaz e envolvente que facilite a aprendizagem prática e progressiva em cursos de graduação em tecnologia.

Palavras-chave: ensino de programação, lógica de programação, jogos educacionais, gamificação, aprendizagem tangencial.

ABSTRACT

This Final Undergraduate Project proposes the development of an educational digital game focused on teaching programming logic to higher education students. The initiative aims to address common learning difficulties faced by beginners, such as abstraction, logical reasoning, and syntax, through a playful and interactive approach. The game incorporates elements of Programming by Demonstration (PBD), tangential learning, and gamification, allowing players to explore fundamental programming concepts without requiring textual code. The project presents an effective and engaging alternative to support practical and gradual learning in undergraduate technology courses.

Keywords: programming education, logic programming, educational games, gamification, tangential learning.

SUMÁRIO

RESUMO ..... **II**

ABSTRACT ..... **III**

LISTA DE FIGURAS ..... **VI**

LISTA DE TABELAS ..... **VII**

LISTA DE QUADROS ..... **VIII**

LISTA DE ABREVIATURAS E SIGLAS ..... **IX**

CAPÍTULO 1 - INTRODUÇÃO **1**

1.1 CONTEXTUALIZAÇÃO 1

1.2 PROPOSTA 2

1.3 OBJETIVOS 3

1.3.1 Objetivos específicos 3

1.3.2 Questão de pesquisa 4

1.4 CONSIDERAÇÕES SOBRE O CAPÍTULO 4

CAPÍTULO 2 - REVISÃO DA LITERATURA **5**

2.1 GAMIFICAÇÃO E JOGOS SÉRIOS NO CONTEXTO DA EDUCAÇÃO SUPERIOR 6

2.2 ANÁLISE DE JOGOS DE APRENDIZAGEM DE PROGRAMAÇÃO 8

2.2.1 CodeMonkey: A Introdução Lúdica e Suas Limitações no Contexto Universitário 9

2.2.2 CodinGame: O Desafio da Complexidade para Iniciantes 11

2.3 QUADRO COMPARATIVO DOS JOGOS ANALISADOS 13

2.4 OUTROS JOGOS RELEVANTES PARA O ENSINO DE PROGRAMAÇÃO 14

2.5 CONSIDERAÇÕES SOBRE O CAPÍTULO: IDENTIFICANDO O GAP E JUSTIFICANDO A PROPOSTA 15

CAPÍTULO 3 - ESPECIFICAÇÃO DO PROJETO (REVISADO) **18**

3.1 PERSONAS 18

3.2 REQUISITOS FUNCIONAIS 19

3.3 ESPECIFICAÇÃO DA SOLUÇÃO 21

CAPÍTULO 4 - VALIDAÇÃO DA FERRAMENTA **23**

4.1 AMBIENTE NECESSÁRIO PARA VALIDAÇÃO 23
4.2 CONDIÇÕES PARA INICIAR A VALIDAÇÃO....24
4.3 PROCEDIMENTOS PARA REALIZAR A VALIDAÇÃO....24
4.4 PROCEDIMENTOS PARA ANALISAR OS RESULTADOS 25

CAPÍTULO 5 - INFORMAÇÕES DO PROJETO....26

5.1 CRONOGRAMA....26
5.2 RISCOS 27
5.3 CONSIDERAÇÕES SOBRE O CAPÍTULO....28

REFERÊNCIAS BIBLIOGRÁFICAS....29

GLOSSÁRIO....31

LISTA DE FIGURAS

Figura 1 - Página inicial do CodeMonkey 9
Figura 2 - Página inicial do CodinGame 11
Figura 3 - Fotografia da Persona 1 (Gerada com inteligência artificial) 18
Figura 4 - Fotografia da Persona 2 (Gerada com inteligência artificial) 19

LISTA DE TABELAS

Tabela 1. Riscos do Projeto. Fonte: os Autores 27

LISTA DE QUADROS

Quadro 1 - Dificuldades enfrentadas por alunos. Fonte: (Martins, 2024) 1
Quadro 2 - Persona 1. Fonte: os Autores..... 18
Quadro 3 - Persona 2. Fonte: os Autores..... 19
Quadro 4 - Requisitos Funcionais. Fonte: os Autores 21
Quadro 5. Especificação do jogo. Fonte: os Autores 22
Quadro 6. Ambiente do Usuário (requisitos mínimos). Fonte: os Autores 23
Quadro 7. Etapas de validação. Fonte: os Autores 25
Quadro 8. Cronograma de desenvolvimento. Fonte: os Autores..... 27

LISTA DE ABREVIATURAS E SIGLAS

SIGLA	SIGNIFICADO
PUCPR	Pontifícia Universidade Católica do Paraná
PBD	Programming by Demonstration
NPC	Non-Playable Character (Personagem Não Jogável)
UI	User Interface (Interface de usuário)
UX	User Experience (Experiência de usuário)
TCC	Trabalho de Conclusão de Curso
RA	Resultado de Aprendizagem
IA	Inteligência Artificial
LMS	Learning Management System (Sistema de Gestão de Aprendizagem)
IDE	Integrated Development Environment (Ambiente de Desenvolvimento Integrado)
JSON	JavaScript Object Notation (Notação de Objetos JavaScript)
MEEGA+	Model for the Evaluation of Educational Games (Modelo para Avaliação de Jogos Educacionais)

CAPÍTULO 1 - INTRODUÇÃO

1.1 Contextualização

O ensino de lógica de programação constitui um elemento essencial na formação em tecnologia, pois não apenas prepara futuros profissionais para criar soluções computacionais, mas também desenvolve competências valiosas como o pensamento lógico e a habilidade de resolver problemas.

Contudo, é um fato bem estabelecido na literatura acadêmica que as disciplinas introdutórias nesta área representam um desafio considerável para muitos alunos, resultando em insucesso acadêmico e abandono dos cursos (Ambrósio, Almeida, Macedo, Santos, & Franco, 2011). Uma revisão sistemática recente realizada por Martins, Marin e Alves (2024), por exemplo, corrobora essa percepção, apontando que a complexidade do tema frequentemente leva a altos índices de reprovação e desistência nos cursos de tecnologia.

O Quadro 1 menciona a quantidade de ocorrências, em trabalhos de autores diversos, de diversas dificuldades enfrentadas por alunos durante o aprendizado de lógica de programação.

Quadro 1 - Dificuldades enfrentadas por alunos. Fonte: (Martins, 2024)

Dificuldade	Quantidade de ocorrências
Falta de conhecimento em conceitos técnicos de programação	13
Dificuldade ao necessitar utilizar raciocínio lógico	12
Não conseguir pensar de forma abstrata	10
Não conseguir resolver problemas	8
Dificuldades ao tentar utilizar linguagens de programação, sintaxe	7
Dificuldades para compreender e desenvolver algoritmos	6

Falta de conhecimento matemático	6

Essa mesma pesquisa destaca que entre as dificuldades mais comuns estão a capacidade de abstração, a aplicação eficaz do raciocínio lógico e a própria formulação de soluções para problemas – todos aspectos fundamentais da programação. Adicionalmente, a dificuldade em dominar a sintaxe específica das linguagens de programação e a consequente desmotivação diante dos obstáculos iniciais são identificados como fatores que agravam este cenário desafiador. A exposição precoce a esses desafios pode levar à frustração, desmotivação e, eventualmente, ao abandono dos estudos. Dessa forma, o sucesso pedagógico depende não apenas do conteúdo, mas também da maneira como ele é apresentado, especialmente para estudantes que ingressam em cursos superiores pouca ou nenhuma familiaridade prévia com o pensamento computacional.

Diante dessa realidade, evidenciada consistentemente por estudos na área, torna-se fundamental buscar e avaliar abordagens pedagógicas alternativas que possam tornar o aprendizado da lógica de programação mais acessível e motivador. O uso de jogos digitais surge, neste contexto, como uma ferramenta com grande potencial pedagógico.

1.2 Proposta

Este trabalho propõe o desenvolvimento de um jogo digital voltado ao ensino de lógica de programação, fundamentado na técnica de *Programming by Demonstration* (PBD). Essa abordagem apresenta benefícios relevantes no contexto de ensino superior, especialmente durante o aprendizado de disciplinas como algoritmos, programação e tópicos correlatos (Ferreira, Gonzaga, & Santos, 2010). Por não exigir a escrita direta de código e utilizar apenas uma interface visual, a PBD favorece a compreensão prática dos conceitos, tornando-se uma ferramenta acessível a iniciantes. Além disso, estudos anteriores (Smith, 2000) indicam que essa metodologia contribui para o desenvolvimento do raciocínio sequencial, lógico e científico, habilidades fundamentais tanto para a formação acadêmica quanto para a qualificação profissional no mercado de tecnologia.

Complementarmente, o jogo adotará uma estratégia de aprendizagem tangencial (Silva, 2016), isso significa que os conceitos de programação serão

integrados de forma natural à narrativa e aos desafios do jogo, em vez de serem apresentados isoladamente. Espera-se que essa contextualização torne o aprendizado mais significativo e aumente o engajamento do aluno, reduzindo a absorção passiva de conteúdo e proporcionando uma experiência divertida e ao mesmo tempo didática. Outro objetivo central é reduzir a carga cognitiva inicial: para isso, o jogo irá abstrair a complexidade da sintaxe textual tradicional, substituindo-a por uma interface visual e interativa. Dessa forma, o estudante pode concentrar-se nos princípios lógicos fundamentais sem o obstáculo inicial da sintaxe rigorosa, frequentemente citada como uma barreira na literatura.

Através da combinação dessas estratégias: PBD, aprendizagem tangencial e abstração de sintaxe em um ambiente lúdico, almeja-se criar uma experiência de aprendizagem mais inclusiva, estimulante e eficaz. O objetivo final é que os estudantes consigam compreender e aplicar os fundamentos da lógica de programação de maneira prática e gradual, contribuindo, assim, para mitigar as dificuldades e reduzir as taxas de insucesso observadas nas fases iniciais dos cursos da área de tecnologia.

1.3 Objetivos

O objetivo geral é desenvolver um jogo digital voltado ao ensino de lógica de programação para estudantes do ensino superior, utilizando a técnica de PBD e estratégias de aprendizagem tangencial, com o intuito de reduzir as dificuldades iniciais relacionadas à abstração, sintaxe e raciocínio lógico.

1.3.1 Objetivos específicos

- Projetar mecânicas de jogo que incorporem elementos da técnica PBD, permitindo a construção de lógica de programação sem a necessidade de código textual.
- Integrar os conceitos de programação à narrativa do jogo de maneira natural, seguindo os princípios da aprendizagem tangencial.
- Reduzir a carga cognitiva inicial, substituindo a sintaxe tradicional por uma interface visual interativa de modo a facilitar o aprendizado.
- Auxiliar o ensino da lógica de programação em cursos superiores voltados a tecnologia.

1.3.2 Questão de pesquisa

Como um jogo que simplifica lógica de programação através de uma narrativa medieval e elementos de magia pode contribuir para o ensino em cursos superiores voltados a tecnologia?

1.4 Considerações sobre o capítulo

Este capítulo apresentou o contexto do problema enfrentado por alunos iniciantes na área de programação, com base em estudos recentes que apontam a complexidade da lógica e da sintaxe como fatores recorrentes de dificuldade. Também foram introduzidas as abordagens pedagógicas que embasam este trabalho: a técnica PBD e a aprendizagem tangencial, ambas voltadas à facilitação e engajamento do processo de ensino e aprendizagem. Por fim, foram definidos o objetivo geral, os objetivos específicos e a questão de pesquisa que guiarão o desenvolvimento do projeto.

CAPÍTULO 2 - REVISÃO DA LITERATURA

Conforme estabelecido na introdução deste trabalho, o aprendizado inicial de programação apresenta desafios consideráveis para estudantes universitários, frequentemente ligados à abstração, raciocínio lógico e à sintaxe das linguagens, resultando em dificuldades e índices preocupantes de insucesso acadêmico. Em resposta a esse cenário, diversas abordagens pedagógicas inovadoras têm sido exploradas. Dentre elas, destacam-se a **Gamificação** – aplicação de elementos de jogos em contextos não lúdicos – e o desenvolvimento de *Serious Games* (Jogos Sérios) – jogos cujo propósito primário é educacional (al., 2011; Abt, 1970; Djaouti et al., 2011). Ambas as estratégias buscam utilizar o potencial inerente dos jogos para aumentar a motivação, o engajamento e facilitar a compreensão de conceitos complexos, como os encontrados na programação.

Este capítulo se aprofunda especificamente na análise de **jogos** que se propõem a ensinar programação. O objetivo é examinar criticamente como algumas das soluções mais conhecidas no mercado implementam princípios lúdicos e pedagógicos, avaliando seus pontos fortes, limitações e, crucialmente, sua adequação para estudantes universitários do primeiro período. Para delinear o espectro de abordagens existentes e identificar lacunas, focaremos nossa análise em dois jogos representativos de extremos distintos: **CodeMonkey**, voltado para uma introdução mais básica e visual, e **CodinGame**, direcionado a desafios mais complexos e a um público com maior conhecimento prévio.

A análise detalhada desses dois jogos permitirá compreender suas mecânicas de jogo, as estratégias de ensino empregadas, as linguagens abordadas e, principalmente, como eles se posicionam em relação às necessidades específicas de estudantes que iniciam sua jornada na programação no ensino superior. Através da identificação das limitações de cada uma para este público – seja pela abordagem potencialmente infantilizada e de escopo restrito (CodeMonkey), seja pela complexidade excessiva e falta de

foco no aprendizado fundamental (CodinGame) – busca-se evidenciar o *gap* que o jogo proposto neste Trabalho de Conclusão de Curso (TCC) pretende preencher. Esta revisão fornecerá, portanto, o embasamento teórico e prático do estado da arte, justificando a relevância da solução a ser desenvolvida e orientando seu design.

2.1 Gamificação e Jogos Sérios no Contexto da Educação Superior

Antes de adentrarmos na análise específica dos jogos de programação, é pertinente contextualizar os conceitos de Gamificação e Jogos Sérios, especialmente sua aplicação no ensino superior. Embora frequentemente usados de forma intercambiável, possuem nuances distintas.

A *Gamificação*, conforme definida por Deterding et al. (2011), refere-se ao "uso de elementos de design de jogos em contextos não lúdicos". No âmbito educacional, isso se traduz na incorporação de mecânicas como pontos, medalhas/conquistas (badges), rankings (leaderboards), desafios, narrativas e sistemas de progressão em atividades de aprendizagem tradicionais. O objetivo principal é aumentar a motivação intrínseca e extrínseca dos alunos, promover o engajamento com o conteúdo, incentivar a participação ativa e fornecer feedback imediato sobre o desempenho. Por exemplo, um sistema de gestão de aprendizagem (LMS) pode ser gamificado para recompensar os alunos por completarem leituras, participarem de fóruns ou entregarem tarefas no prazo.

Por outro lado, os *Serious Games* (Jogos Sérios), termo popularizado por Ben Sawyer, são jogos cujo propósito principal vai além do puro entretenimento (Abt, 1970; Djaouti et al., 2011). Eles são projetados com um objetivo específico, como educação, treinamento, simulação, conscientização social ou saúde. No contexto educacional, um Jogo Sério é uma aplicação completa, um ambiente lúdico e interativo onde o aprendizado ocorre através da própria jogabilidade. Os jogadores aprendem ao enfrentar desafios, tomar decisões, experimentar consequências e interagir com o sistema do jogo, que encapsula o conhecimento ou a habilidade a ser adquirida. O aprendizado torna-se, idealmente, uma consequência natural e integrada da experiência de jogar.

A aplicação dessas abordagens no ensino superior, particularmente em áreas como Ciência da Computação e Engenharia, onde a programação é

fundamental, mostra-se promissora. Disciplinas introdutórias de programação frequentemente lidam com altas taxas de evasão e reprovação, atribuídas à dificuldade inicial e à percepção de aridez do conteúdo (Bennedsen & Caspersen, 2007; Martins, Marin e Alves, 2024). A gamificação e os jogos sérios podem atuar diretamente sobre esses fatores:

- **Aumento da Motivação e Engajamento:** Elementos como narrativas envolventes, desafios progressivos e recompensas podem transformar a percepção do aluno sobre a tarefa de aprender a programar, tornando-a menos intimidante e mais atrativa (Hamari et al., 2014). A falta de motivação é um fator que agrava as dificuldades e contribui para o baixo desempenho (Martins, Marin e Alves, 2024).
- **Feedback Imediato e Contínuo:** Jogos são excelentes em fornecer feedback instantâneo. No aprendizado de programação, isso é crucial. Um jogo pode indicar imediatamente se um trecho de código funciona, por que falhou e como corrigi-lo, acelerando o ciclo de aprendizado por tentativa e erro (Gee, 2003).
- **Ambiente Seguro para Experimentação:** O medo de errar pode inibir a experimentação, essencial no aprendizado de programação. Jogos criam um espaço seguro onde falhar faz parte do processo de aprendizado, sem as mesmas consequências de um erro em uma avaliação tradicional.
- **Visualização de Conceitos Abstratos:** Jogos podem tornar conceitos abstratos de programação (como loops, condicionais, variáveis, estruturas de dados) mais concretos e visuais, facilitando a compreensão inicial (Papert, 1980). A dificuldade com a abstração é uma das mais citadas na literatura como barreira para aprender a programar (Martins, Marin e Alves, 2024).
- **Desenvolvimento de Habilidades de Resolução de Problemas:** Muitos jogos de programação são estruturados em torno de desafios que exigem decomposição do problema, planejamento lógico e aplicação de conceitos de programação para encontrar a solução, habilidades essenciais para qualquer programador. O desenvolvimento do raciocínio lógico e da capacidade de resolver problemas são

> frequentemente apontados como os maiores desafios para os iniciantes (Martins, Marin e Alves, 2024).

Contudo, a implementação eficaz de gamificação e jogos sérios não é isenta de desafios. Um design inadequado pode levar à chamada "gamificação superficial" (chocolate-covered broccoli), onde elementos de jogo são apenas adicionados superficialmente sem uma integração significativa com os objetivos pedagógicos, resultando em baixo impacto ou até mesmo distração (Nicholson, 2015). Além disso, o desenvolvimento de jogos sérios de alta qualidade exige expertise em design de jogos, pedagogia e no domínio específico (neste caso, programação), tornando o processo complexo e custoso. É fundamental que o design seja centrado no aprendizado, garantindo que a diversão e o engajamento estejam a serviço da aquisição de conhecimento e habilidades relevantes.

A análise dos jogos a seguir busca entender como esses princípios e desafios se manifestam na prática, focando nos extremos do espectro para identificar o espaço para inovação.

2.2 Análise de Jogos de Aprendizagem de Programação

Diversos jogos surgiram com a proposta de ensinar programação de forma lúdica. Para entender o cenário e identificar oportunidades, analisaremos dois jogos que se destacam em extremos opostos quanto ao público e complexidade: CodeMonkey e CodinGame.



Figura 1 - Página inicial do CodeMonkey

2.2.1 CodeMonkey: A Introdução Lúdica e Suas Limitações no Contexto Universitário

- **Visão Geral e Foco:** CodeMonkey é um jogo educacional reconhecida por sua abordagem amigável e visualmente atraente, claramente direcionado ao público infanto-juvenil (ensino fundamental e médio). Sua premissa é desmistificar a programação através de puzzles simples onde o jogador controla um macaco para coletar bananas, escrevendo código para comandar suas ações.
- **Mecânica e Pedagogia:** O jogo introduz conceitos básicos de programação de forma muito gradual, começando frequentemente com interfaces de blocos que transitam suavemente para código textual, primariamente em CoffeeScript (uma linguagem que compila para JavaScript) e, mais recentemente, adicionando Python. Essa abordagem é eficaz para reduzir a ansiedade inicial e familiarizar os mais jovens com a lógica de programação. O feedback é imediato e visual, reforçando a conexão entre código e ação.
- **Pontos Fortes:** Sua maior força reside na interface intuitiva e no design lúdico que engaja crianças e adolescentes. A progressão lenta e o feedback constante são adequados para quem nunca teve contato com programação.
- **Limitações e Inadequação ao Ensino Superior:** Apesar de seus méritos para o público-alvo original, CodeMonkey apresenta limitações significativas quando considerado para estudantes universitários do primeiro período:
    - **Design "Infantilizado":** A estética geral, os personagens e a narrativa são claramente voltados para crianças. Esse design, embora eficaz

para o público mais jovem, pode ser percebido como "infantilizado" por estudantes universitários, gerando desinteresse ou até mesmo constrangimento, o que desencoraja a adoção e o engajamento por parte de adultos ingressantes no ensino superior.

- **Linguagem de Foco Inicial:** O uso primário de CoffeeScript, embora facilite a transição visual-texto, não representa uma linguagem de alta demanda ou comumente ensinada como primeira linguagem em cursos universitários de computação no Brasil, que frequentemente preferem Python, Java ou C. Embora os conceitos lógicos sejam transferíveis, o aprendizado direto em uma linguagem menos relevante pode ser visto como um desvio ineficiente do currículo padrão.
- **Profundidade Limitada:** O jogo foca nos conceitos mais básicos. Embora introduza variáveis, loops e condicionais, a profundidade e a complexidade dos problemas raramente atingem o nível esperado para consolidar o conhecimento necessário para disciplinas universitárias subsequentes. Falta a abordagem de estruturas de dados mais complexas, paradigmas de programação ou algoritmos mais elaborados, que são fontes comuns de dificuldade para alunos universitários (Martins, Marin e Alves, 2024).
- **Modelo de Negócio:** Ser um jogo predominantemente pago também pode restringir seu uso em larga escala no contexto universitário público.

Em resumo, CodeMonkey serve como um excelente exemplo de como tornar a introdução à programação acessível e divertida para crianças, mas seu design, escolha de linguagem inicial e profundidade limitada o tornam inadequado como ferramenta principal para estudantes universitários do primeiro período, que necessitam de uma abordagem mais madura, profunda e alinhada às linguagens e desafios do ensino superior.

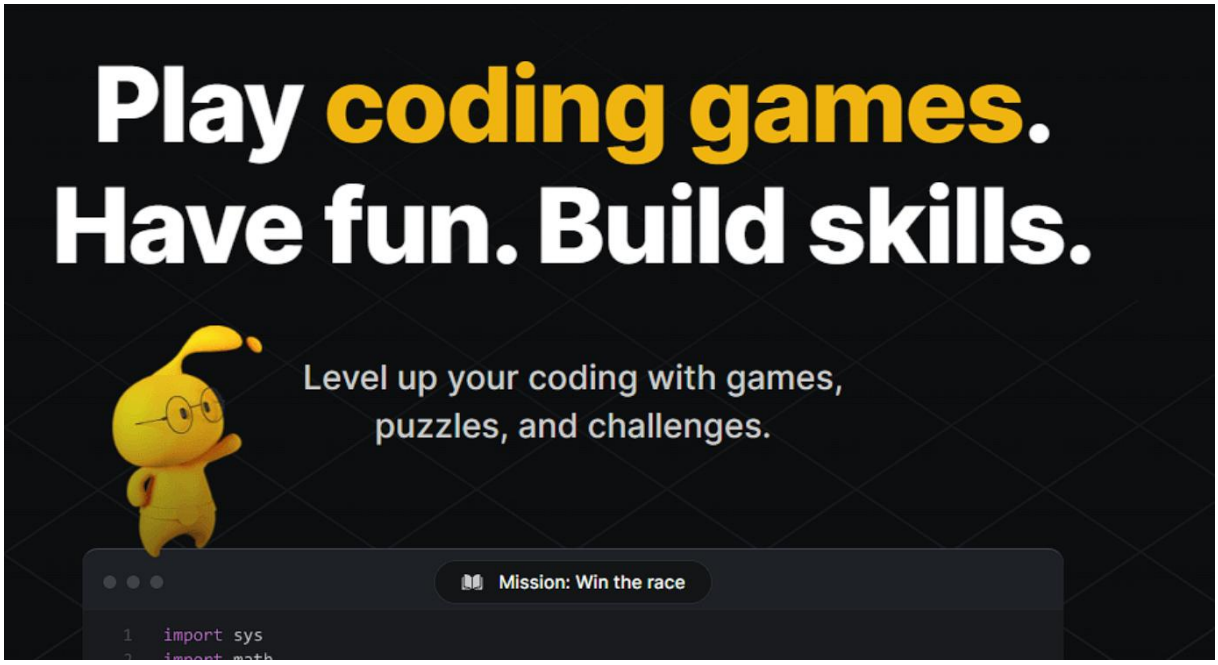


Figura 2 - Página inicial do CodinGame

2.2.2 CodinGame: O Desafio da Complexidade para Iniciantes

- **Visão Geral e Foco:** No extremo oposto do espectro, encontramos CodinGame. Este jogo não se limita a iniciantes; ele atrai programadores de níveis intermediário e avançado, oferecendo um ambiente para aprimorar habilidades, aprender novas linguagens e competir em desafios complexos que simulam problemas de desenvolvimento de software do mundo real.
- **Mecânica e Pedagogia:** CodinGame utiliza minijogos como contexto para problemas de programação. Frequentemente, o usuário não apenas controla um elemento, mas desenvolve IAs simples, otimiza algoritmos ou soluciona puzzles lógicos e matemáticos. Seu grande diferencial é o suporte a mais de 25 linguagens de programação, permitindo grande flexibilidade. O jogo promove um ambiente competitivo com rankings e desafios multiplayer, focando na eficiência e otimização do código.
- **Pontos Fortes:** A vasta gama de linguagens, a complexidade e variedade dos desafios, e o forte senso de comunidade e competição são seus maiores trunfos. É uma ferramenta excelente para programadores que já possuem uma base sólida e buscam aprofundamento, prática com algoritmos complexos e exposição a diferentes paradigmas e linguagens.

- **Limitações e Inadequação ao Ensino Superior (Iniciantes):** Para estudantes universitários do primeiro período, especialmente aqueles sem experiência prévia significativa, CodinGame apresenta barreiras consideráveis:
    - **Curva de Aprendizado Íngreme:** O jogo, mesmo em seus níveis iniciais, pressupõe uma familiaridade com conceitos de programação e pensamento algorítmico que muitos ingressantes ainda não possuem. A interface e a natureza abstrata de muitos desafios podem ser intimidadoras e frustrantes para quem está apenas começando a aprender o que é uma variável ou um loop. A literatura aponta que a dificuldade em aplicar raciocínio lógico e abstração é um dos principais entraves iniciais (Martins, Marin e Alves, 2024).
    - **Foco em Desempenho e Otimização:** Enquanto universidades buscam primeiramente ensinar os fundamentos da lógica, sintaxe e boas práticas, CodinGame rapidamente direciona o foco para a otimização e a competição, o que pode ser prematuro e desviar a atenção do aprendizado conceitual básico necessário no início.
    - **Ausência de Roteiro Pedagógico Estruturado para Iniciantes:** Diferente de jogos focados no ensino introdutório, CodinGame funciona mais como um "playground" ou uma "academia" para programadores. Falta um caminho pedagógico claramente definido e sequencial para guiar um iniciante absoluto desde os conceitos mais básicos até os mais complexos de forma sistemática e didática. O aprendizado depende muito da iniciativa e capacidade de autodireção do usuário em navegar pela vasta gama de desafios.
    - **Barreira Linguística:** O jogo opera primariamente em inglês e francês, o que pode dificultar o acesso e a compreensão para estudantes brasileiros com limitações nesses idiomas, conforme também apontado como uma dificuldade geral por Pontes (2013) citado em Martins, Marin e Alves (2024).

Portanto, CodinGame, apesar de ser uma ferramenta poderosa para praticar e aprofundar conhecimentos, mostra-se inadequada como um jogo para o

ensino introdutório de programação no primeiro período universitário. Sua complexidade, foco em otimização e falta de um percurso pedagógico estruturado para iniciantes a tornam mais apropriada para etapas posteriores da formação ou para estudantes com conhecimento prévio.

2.3 Quadro Comparativo dos Jogos Analisados

Característica	CodeMonkey	CodinGame
Público Principal	Crianças,adolescentes, iniciantes *absolutos*	Iniciantes (motivados) a profissionais experientes
Abordagem	Puzzles visuais, controle direto de avatar	Puzzles/Jogos complexos, IA, otimização, competição
Linguagens Foco	CoffeeScript(inicial),Python, HTML5	+25 (Python, Java, C++, JS, C#, Ruby, etc.)
Curva Aprendizado	Muito suave	Íngreme (para iniciantes)
Foco Pedagógico	Introdução à lógica, sintaxe básica (visual)	Algoritmos, estruturas dados, resolução problemas comp.
Feedback	Imediato, visual	Testes automatizados, output do código, rankings
Engajamento	Alto (visual), **percepção infantilizada**	Alto (desafios, competição), menos guiado
Disponibilidade PT	Não (Inglês, Espanhol)	Não (Inglês, Francês)
Modelo	Pago (com trial)	Gratuito (maioria dos puzzles/competições)

Relevância 1º Per.	Baixa (inadequado p/ público e profundidade)	Baixa (principal), Alta (complementar p/ avançados)

2.4 Outros Jogos Relevantes para o Ensino de Programação

Além dos jogos CodeMonkey e CodinGame analisados em detalhe no Capítulo 2, um mapeamento sistemático da literatura recente identificou diversas outras iniciativas de jogos digitais voltados ao ensino de lógica e conceitos de programação, publicados em eventos científicos nacionais relevantes entre 2017 e 2021. A seguir, apresentam-se brevemente cinco exemplos dessas ferramentas, ilustrando a variedade de abordagens, públicos e tecnologias empregadas na área:

1. **POOkémon:** Este jogo foi desenvolvido com foco específico no ensino de conceitos de Programação Orientada a Objetos (POO). Utilizando a linguagem Java, a ferramenta direciona-se principalmente a estudantes do ensino superior. A avaliação mencionada no estudo que o analisou focou na motivação dos usuários.
2. **Fluxogame:** Trata-se de um jogo projetado para auxiliar no aprendizado de algoritmos e lógica de programação através de fluxogramas. A proposta indica inspiração nas linguagens Portugol e C, possivelmente adaptando a lógica para um formato de jogo de cartas. Foi aplicado tanto no ensino médio/técnico quanto no superior.
3. **Projeto Éden:** Caracterizado como um Jogo Sério, esta ferramenta tem como objetivo o ensino de variáveis e tipos de dados. Desenvolvido com a ferramenta Construct 2, foi aplicado em contextos de ensino médio/técnico e superior.
4. **Looking for Pets:** Este jogo mobile (Android) foca na criação de algoritmos como meio para desenvolver o Pensamento Computacional. Embora direcionado ao ensino fundamental/médio no estudo mapeado, sua abordagem algorítmica pode ser relevante. Foi um dos poucos jogos mencionados como potencialmente disponível para uso e avaliou aspectos como estética, aprendizagem e desafio.

5. **SQL Planet:** Diferenciando-se por focar em bancos de dados, este é um jogo online projetado especificamente para o ensino da linguagem SQL (Structured Query Language). O público-alvo são pessoas que desejam aprimorar conhecimentos em SQL.

Estes exemplos adicionais ilustram a variedade de abordagens lúdicas existentes para o ensino de programação e conceitos relacionados. Embora representem iniciativas relevantes e demonstrem a amplitude de aplicações possíveis, não foram incluídos na análise central do Capítulo 2, pois o foco foi o contraste entre os extremos de simplicidade (CodeMonkey) e complexidade (CodinGame) para melhor definir o *gap* específico – um espaço onde as ferramentas atuais se mostram inadequadas para o público universitário iniciante – que o jogo proposto neste TCC busca preencher.

2.5 Considerações sobre o Capítulo: Identificando o Gap e Justificando a Proposta

A análise focada em CodeMonkey e CodinGame expõe claramente um *gap* significativo no cenário de jogos para aprender programação, especialmente quando consideramos as necessidades dos estudantes universitários do primeiro período no Brasil. De um lado, temos CodeMonkey, um jogo que acerta na abordagem lúdica e na suavidade da introdução, mas falha em adequar-se ao público universitário devido ao seu design percebido como infantilizado, à escolha de uma linguagem inicial menos relevante (CoffeeScript) e à limitada profundidade conceitual para os objetivos acadêmicos, sendo muito básico.

Do outro lado, temos CodinGame, um jogo poderoso e desafiador, com vasta aplicabilidade para programadores que já possuem alguma base. Contudo, sua alta complexidade, curva de aprendizado íngreme, foco em otimização e competição, e a ausência de um roteiro pedagógico estruturado para iniciantes a tornam inadequada, e muitas vezes intimidante, para a maioria dos estudantes que estão tendo seu primeiro contato formal com a programação na universidade. Ele é muito avançado ou desfocado para o aprendizado inicial. Esta dificuldade inicial é corroborada por estudos como o de Martins, Marin e

Alves (2024), que destacam o raciocínio lógico e a abstração como os principais desafios levando frequentemente a altas taxas de reprovação e desistência.

É precisamente neste espaço – entre o excessivamente básico/infantil e o excessivamente complexo/avançado – que o jogo proposto neste TCC se insere. Existe uma clara necessidade de uma solução que:

- **Seja Adequada ao Público Universitário**: Apresente um design, narrativa e tom que engajem jovens adultos, evitando a percepção de ser infantil, mas mantendo o caráter lúdico e motivador.
- **Utilize Linguagens Relevantes**: Foque desde o início em linguagens amplamente utilizadas no contexto acadêmico e profissional brasileiro, como Python ou JavaScript, permitindo aplicação direta no curso.
- **Ofereça uma Curva de Aprendizado Apropriada**: Comece com conceitos básicos de forma acessível, mas progrida de maneira consistente e estruturada para cobrir o conteúdo fundamental esperado em uma disciplina introdutória de programação universitária, incluindo lógica, estruturas de controle, funções, estruturas de dados básicas etc., abordando diretamente as dificuldades técnicas mais comuns identificadas na literatura (Martins, Marin e Alves, 2024).
- **Mantenha o Foco Pedagógico**: Utilize as mecânicas de jogo para reforçar o aprendizado dos fundamentos da programação e do pensamento computacional, sem desviar excessivamente para a competição ou otimização prematura. O foco deve ser auxiliar no desenvolvimento do raciocínio lógico e capacidade de abstração.
- **Seja Acessível**: Idealmente, seja oferecida em português e com um modelo de acesso que facilite sua adoção por estudantes e instituições brasileiras.

Ao mirar neste gap, o jogo proposto não será apenas mais uma ferramenta, mas uma solução direcionada a uma necessidade real e não atendida pelas opções mais proeminentes disponíveis nos extremos do mercado. Ele buscará combinar a acessibilidade e o engajamento inicial inspirados nas abordagens mais simples, com a profundidade conceitual e a relevância prática necessárias para o sucesso no ensino superior, algo que falta nos jogos analisados quando aplicadas a este público específico.

Em suma, a revisão da literatura, focada em CodeMonkey e CodinGame e enriquecida com os achados da revisão sistemática de Martins, Marin e Alves (2024) sobre os desafios persistentes na aprendizagem de programação, valida a relevância e a oportunidade para o desenvolvimento do jogo proposto neste trabalho. As lições aprendidas sobre os pontos fortes e, principalmente, as limitações desses jogos, informam diretamente os requisitos e as decisões de design que serão detalhados no próximo capítulo, visando criar uma ferramenta pedagógica eficaz, motivadora e sob medida para os desafios do aprendizado de programação no início da jornada universitária.

CAPÍTULO 3 - ESPECIFICAÇÃO DO PROJETO

3.1 Personas

Para orientar o design pedagógico e técnico do jogo proposto, foram definidas personas que representam os perfis de estudantes que enfrentam dificuldades no aprendizado de lógica de programação em cursos superiores da área de tecnologia. As personas foram criadas com base em entrevistas informais com estudantes da PUCPR.

Persona 1	Gabriel, 19 anos



Figura 3 - Fotografia da Persona 1 (Gerada com inteligência artificial)

Curso	Engenharia de Software
Semestre	1º
Objetivo	Acompanhar bem as disciplinas básicas e não depender de monitorias
Frustrações	Dificuldade em entender lógica de algoritmos e baixa familiaridade com programação
Habilidades	Aprende bem com exemplos visuais e jogos, mas sente desmotivação em aulas expositivas
Interesses	Gosta de jogos, tecnologia e cultura geek

Quadro 2 - Persona 1. Fonte: os Autores

Persona 2	Júlia, 22 anos



Figura 4 - Fotografia da Persona 2 (Gerada com inteligência artificial)

Curso	Análise e Desenvolvimento de Sistemas
Semestre	2º
Objetivo	Sentir-se mais segura para construir seus próprios scripts e automatizações
Frustrações	Tem dificuldade com estruturas condicionais e loops, e frequentemente confunde operadores lógicos
Habilidades	Boa compreensão visual e lógica, gosta de narrativas e aprende melhor com prática
Interesses	UX, desenvolvimento web, livros de fantasia

Quadro 3 - Persona 2. Fonte: os Autores

Essas personas refletem dois perfis comuns de estudantes iniciantes: aqueles que se frustram com a falta de conexão entre teoria e prática, e aqueles que possuem potencial cognitivo, mas encontram barreiras na sintaxe e na abstração. O projeto do jogo busca atender ambos os perfis, oferecendo uma experiência de aprendizagem prática, contextualizada e lúdica.

3.2 Requisitos Funcionais

Com base nas necessidades observadas durante a definição das personas e no problema pedagógico apresentado, a seguinte lista de requisitos funcionais compõe a primeira versão do escopo do projeto:

ID	Requisito
RF01	O jogador deve ser capaz de mover seu personagem horizontalmente (esquerda e direita).
RF02	O jogador deve ser capaz de pular e interagir com plataformas utilizando física 2D (gravidade, colisão).

RF03	O jogo deve detectar colisões entre o jogador e o ambiente, limitando o movimento conforme obstáculos.
RF04	O jogo deve conter elementos interativos no cenário, como portas, alavancas ou plataformas móveis.
RF05	O jogo deve apresentar obstáculos físicos (como buracos ou espinhos) que causem a morte do jogador.
RF06	O jogo deve conter checkpoints e/ou sistemas de salvamento automático por progresso em cada capítulo.
RF07	O sistema deve reiniciar o jogador no último checkpoint em caso de morte.
RF08	O jogador deve ser capaz de interagir com elementos por proximidade ou ação (como pressionar um botão).
RF09	O jogador deve poder interagir com objetos do jogo para visualizar e editar propriedades expostas.
RF10	O sistema deve refletir em tempo real as alterações feitas nas variáveis de objetos do jogo.
RF11	O jogo deve apresentar desafios que exigem compreensão e manipulação de variáveis para serem superados.
RF12	O jogo deve conter desafios onde o jogador precisa usar operadores aritméticos em variáveis numéricas.
RF13	O jogo deve conter desafios que envolvam o uso de operadores relacionais e lógicos para tomada de decisão.
RF14	O jogador deve ser capaz de modificar estruturas condicionais (if/else) através de uma interface acessível.
RF15	O jogador deve interagir com estruturas de repetição (como loops) para alterar comportamentos repetitivos no jogo.
RF16	O jogo deve apresentar situações que envolvam vetores, permitindo ao jogador alterar ou acessar índices.
RF17	O jogo deve apresentar situações que envolvam o uso de matrizes em elementos visuais ou lógicos.
RF18	O jogo deve ser dividido em capítulos temáticos, cada um focado em um ou mais conceitos de lógica.
RF19	O sistema deve permitir o início de diálogos interativos com NPCs com

	ramificações e múltiplas escolhas.
RF20	O jogador deve desbloquear novas áreas ou funcionalidades ao completar desafios lógicos.
RF21	O sistema deve oferecer uma ferramenta visual (“ferramenta mágica”) que represente o ato de editar o mundo.
RF22	O jogo deve aplicar efeitos visuais e narrativos para reforçar momentos importantes da progressão.
RF23	O jogo deve apresentar uma estrutura de missões que guiem o jogador na aplicação dos conceitos de lógica.
RF24	O sistema deve impedir alterações não permitidas e validar os dados inseridos pelo jogador.
RF25	O jogo deve incluir feedbacks visuais e/ou auditivos que indiquem o sucesso ou falha na execução das ações.
RF26	O jogo deve fornecer uma introdução contextual (narrativa) que motive o jogador a explorar as mecânicas.

Quadro 4 - Requisitos Funcionais. Fonte: os Autores

3.3 Especificação da solução

Nome do Jogo	Crônicas de Bugdônia
Tecnologias utilizadas	Godot Engine, GDScript, Git, Azure DevOps, Krita
Tema da Engenharia de Software	Lógica de programação: Algoritmos, estruturas de controle, estruturas de dados homogêneos, funções e recursividade

História do Jogo

O jogador assume o papel de um cavaleiro preso injustamente nas masmorras do Reino de Bugdônia, uma terra medieval amaldiçoada por bugs lógicos. Após encontrar por acaso uma varinha mágica capaz de “editar a realidade”, ele descobre que tudo no mundo é regido por variáveis, funções e lógica. Com essa nova habilidade, ele começa sua jornada para escapar da prisão, explorando os capítulos do reino, onde enfrentará desafios crescentes que envolvem raciocínio lógico e manipulação de estruturas como condicionais, laços de repetição, vetores e matrizes. Seu objetivo final é enfrentar o terrível Dragão Elseifor com o objetivo de resgatar a Princesa Arraya.

Objetivos do Jogo

- Progredir por um mundo repleto de obstáculos interativos que simulam situações lógicas do cotidiano da programação.
- Resolver problemas de complexidade crescente que exigem a compreensão e manipulação de variáveis, condições, repetições e estruturas de dados.
- Usar a ferramenta mágica para interagir com o mundo de forma prática, editando propriedades e comportamentos de objetos para avançar nas missões.
- Compreender gradualmente os fundamentos da lógica computacional ao superar desafios que envolvem desde operações simples até raciocínio estruturado com vetores e funções.
- Desenvolver autonomia na resolução de problemas ao interpretar pistas narrativas, missões e consequências de suas ações.

Inspirações

- **Scribblenauts**: pela forma criativa de interagir com o mundo e resolver problemas, dando ao jogador liberdade para solucionar desafios da maneira que quiser.
- **Noita**: por seu sistema de feitiços modulares, que se assemelha a conceitos de programação funcional e encadeamento de comandos.
- **Magicraft**: por adotar um sistema de feitiços modulares semelhante ao de Noita, mas com uma abordagem mais acessível e intuitiva, permitindo que jogadores casuais compreendam e se divirtam com lógica de encadeamento sem a complexidade excessiva.
- **Portal**: como referência para a construção de puzzles integrados à narrativa, promovendo aprendizagem emergente dentro do fluxo do jogo.
- **CodeMonkey**: oferece um bom ponto de partida para iniciantes, mas apresenta uma abordagem muito linear e simplificada, o que pode limitar a exploração livre de conceitos e reduzir o engajamento com o conteúdo.
- **CodinGame**: propõe desafios mais realistas e técnicos, mas frequentemente se torna competitivo e denso demais para iniciantes absolutos, afastando aqueles que ainda não dominam os fundamentos da lógica de programação.

ID	Resultado de Aprendizagem	Tema de Estudo	Mecânica do Jogo
RA1	Compreender e aplicar conceitos fundamentais de lógica de programação para solucionar problemas.	Variáveis, Operadores, Estruturas de Controle e Repetição	Jogador edita variáveis e estruturas de controle usando a varinha mágica para alterar o mundo.
RA2	Utilizar estruturas de dados e funções simples para resolver interações e comportamentos lógicos.	Vetores, Matrizes e Funções	Jogador combina feitiços (funções) e manipula vetores/matrizes para resolver desafios e puzzles.

Quadro 5. Especificação do jogo. Fonte: os Autores

CAPÍTULO 4 - VALIDAÇÃO DA FERRAMENTA

A validação da ferramenta desenvolvida tem como objetivo verificar se os objetivos educacionais propostos foram alcançados de maneira eficaz, utilizando métodos formais de análise e um processo sistemático de avaliação. Para isso, adotaremos o modelo MEEGA+ (Model for the Evaluation of Educational Games), proposto por Petri, von Wangenheim e Borgatto (2018), amplamente reconhecido na área de jogos educacionais.

4.1 Ambiente necessário para validação

A fim de facilitar o acesso e a execução do jogo, a ferramenta será exportada para Web e hospedada em um servidor acessível via navegador. Essa abordagem garante maior compatibilidade e independência de sistema operacional, permitindo que os alunos realizem a validação diretamente em seus próprios dispositivos, inclusive nos laboratórios da universidade. Abaixo estão os requisitos mínimos do ambiente do usuário:

Dispositivo	Notebook ou computador
Sistema operacional	Windows, Linux ou macOS
Navegador	Firefox 79, Chrome 68, Edge 79, Safari 15.2 ou Opera 64
Requisitos gráficos	Suporte a WebGL 2.0 (presente na maioria dos navegadores modernos)
Memória RAM	4GB
Conexão com internet	Necessário para carregamento inicial do jogo

Quadro 6. Ambiente do Usuário (requisitos mínimos). Fonte: os Autores

Essa abordagem garante uma validação prática e escalável, eliminando a necessidade de instalação local do jogo.

4.2 Condições para iniciar a validação

Antes de dar início à validação, serão realizados os seguintes preparativos:

- Publicação do jogo em uma plataforma de hospedagem acessível por link.
- Criação de um formulário de avaliação estruturado, baseado no modelo MEEGA+, para coletar feedback dos participantes.
- Seleção dos participantes: mínimo de 10 alunos de cursos de graduação da área de computação que estão cursando ou já cursaram disciplinas de lógica ou raciocínio algorítmico.
- Consentimento livre e esclarecido, garantindo o uso ético das respostas coletadas.

4.3 Procedimentos para realizar a validação

A validação será realizada de forma presencial em sala, com computadores e internet ou remotamente, de acordo com a disponibilidade da instituição. A validação seguirá as etapas abaixo:

Abertura	Apresentação do projeto e explicação dos objetivos da avaliação.
Execução do jogo	Os participantes terão acesso à versão web do jogo por tempo indeterminado, com liberdade para explorar os capítulos do jogo. Durante essa etapa, nenhuma ajuda adicional será oferecida, de forma a não interferir na percepção individual de aprendizado e usabilidade.
Preenchimento do formulário	Após a sessão de jogo, os alunos preencherão um questionário digital (formulário estruturado com base no modelo MEEGA+) que abordará aspectos como: • Clareza da proposta educacional (entendimento dos objetivos do jogo) • Engajamento e motivação (nível de interesse, satisfação e imersão durante a experiência) • Qualidade da interface e usabilidade (facilidade de uso,

	navegação e clareza das instruções) • Aprendizagem percebida (o quanto o aluno acredita ter aprendido com o jogo)

Quadro 7. Etapas de validação. Fonte: os Autores

Caso a avaliação ocorra presencialmente, também será possível observar comportamentos espontâneos, como expressões faciais, comentários verbais e dificuldades técnicas. Essas observações serão registradas como apoio qualitativo à análise, contribuindo para uma compreensão mais completa da experiência do aluno com o jogo.

4.4 Procedimentos para analisar os resultados

A análise dos resultados será baseada nas diretrizes do modelo MEEGA+, considerando três principais dimensões:

- **Engajamento:** grau de motivação, interesse e envolvimento do aluno durante o uso do jogo.
- **Aprendizagem percebida:** compreensão declarada dos conceitos abordados no jogo, segundo a percepção dos participantes.
- **Qualidade do jogo:** inclui aspectos como clareza das instruções, usabilidade, interatividade e adequação da proposta educacional.

Os dados quantitativos (respostas ao questionário) serão organizados em gráficos e tabelas, permitindo identificar padrões. Já os comentários abertos (qualitativos) serão analisados por categorias, auxiliando na identificação de melhorias e pontos fortes.

Ao final, será feita uma correlação entre os objetivos pedagógicos estabelecidos e os resultados obtidos, de modo a verificar se o jogo atendeu suas metas educacionais e pode ser utilizado como ferramenta complementar de ensino.

CAPÍTULO 5 - INFORMAÇÕES DO PROJETO

5.1 Cronograma

O desenvolvimento do projeto será organizado em sprints semanais, com acompanhamento por meio do Azure Boards. As tarefas foram planejadas considerando uma progressão incremental e modular, começando pela construção da base técnica do jogo, seguida pela implementação das mecânicas educacionais, narrativa e validações.

Sprint	Período	Tarefas previstas
01	XX/XX/2025 XX/XX/2025	Estrutura inicial do projeto, testes com engine escolhida
02		Implementação da movimentação básica, física do jogador, sistema de colisões, plataformas e obstáculos
03		Implementação de sistema de variáveis e interface de edição in-game
04		Integração da edição em tempo real com lógica de jogo e feedback visual
05		Implementação do sistema de diálogos com NPCs com múltiplas opções
06		Implementação de sistema de missões
07		Desenvolvimento dos primeiros desafios baseados em tipos de dados e variáveis
08		Sistema de operadores aritméticos e lógicos com desafios interativos
09		Implementação de manipulação de funções e loops
10		Criação de desafios com loops e comportamentos repetitivos
11		Implementação de acesso/edição de

	vetores e matrizes
12	Desenvolver desafios para vetores e matrizes
13	Desenvolver sistema de mapa por capítulos
14	Revisar narrativa e adicionar objetivos adicionais
15	Melhorar a estética do jogo, criar mais objetos de fundo etc.
16	Polimento e ajustes finais

Quadro 8. Cronograma de desenvolvimento. Fonte: os Autores

5.2 Riscos

Tabela 1. Riscos do Projeto. Fonte: os Autores

Id	Descrição	P	I	S	Ação de Prevenção	Ação de Contingência
R01	Complexidade na criação do editor visual de variáveis	2	2	4	Iniciar com uma prova de conceito mínima	Reduzir escopo para um editor limitado apenas a certos tipos de dados
R02	Baixo conhecimento da equipe sobre a engine/ferramenta escolhida	3	3	9	Realizar estudos e protótipos paralelos	Reutilizar templates e limitar o escopo técnico
R03	Dificuldade de aplicar os conceitos de lógica de forma divertida	2	3	6	Consultar exemplos de jogos educativos	Simplificar os desafios com feedbacks mais guiados
R04	Dificuldade de balancear a curva de aprendizado dos conceitos	3	2	6	Definir um roteiro pedagógico progressivo	Ajustar os desafios conforme o feedback dos jogadores
R05	Problemas de desempenho com renderização ou lógica	2	3	6	Testar otimizações desde os protótipos iniciais	Otimizar elementos gráficos e simplificar a lógica de jogo
R06	Falta de tempo para implementar todas as mecânicas planejadas	3	3	9	Priorizar requisitos essenciais desde o início	Cortar recursos não críticos e modularizar o desenvolvimento
R07	Dificuldade na criação e manutenção de diálogos ramificados	2	2	4	Criar sistema de diálogos baseado em JSON	Restringir complexidade narrativa e manter diálogos mais diretos
R08	Desmotivação da equipe durante longas fases do projeto	2	2	4	Reuniões de alinhamento e revisão dos objetivos	Redefinir escopo e dividir tarefas menores para manter engajamento
R09	Perda de arquivos ou falhas na organização do código	1	3	3	Uso de controle de versão (Git)	Recuperar última versão estável do repositório
R10	Dificuldade de validar o impacto pedagógico sem aplicar testes frequentes com alunos	2	3	6	Realizar simulações internas com equipe ou colegas	Solicitar feedback a professores de lógica de programação
R11	Bugs críticos em funcionalidades básicas (movimento, física)	2	3	6	Criar testes simples e validar funcionalidades centrais constantemente	Corrigir com prioridade e congelar novas features

R12	Dificuldade em controlar permissões de edição pelo jogador	2	2	4	Implementar sistema de validação robusto	Bloquear edições perigosas
R13	Inconsistência entre narrativa e aprendizado lógico	2	2	4	Planejar roteiro e capítulos em conjunto com o design pedagógico	Adaptar narrativa para encaixar com os conceitos
R14	Sobrecarga cognitiva do jogador ao apresentar muitos conceitos de uma vez	3	2	6	Dividir os capítulos de forma modular e gradual	Inserir ajuda contextual e simplificar objetivos

Legenda P: probabilidade (3 = alta, 2 = média, 1 = baixa)
I: impacto (3 = alto, 2 = médio, 1 = baixo)
S: severidade (S=P*I)

5.3 Considerações sobre o Capítulo

Este capítulo apresentou o cronograma das atividades previstas para o desenvolvimento do projeto, distribuídas em sprints, além do mapeamento dos riscos com suas respectivas ações preventivas e contingenciais. Esse planejamento permite maior controle sobre a execução do projeto e oferece estratégias para lidar com possíveis imprevistos, assegurando sua viabilidade técnica e acadêmica.

REFERÊNCIAS BIBLIOGRÁFICAS

ABT, C. C. Serious Games. New York: Viking Press, 1970. Disponível em: https://archive.org/details/seriousgames0000abtc. Acesso em: 08 abr. 2025.

ALVAREZ, J.; DAMIEN, D. An introduction to Serious Game – Definitions and concepts. In: SERIOUS GAMES & SIMULATION WORKSHOP, Paris, 2011. Proceedings... Paris, 2011. p. 10-15. Disponível em: http://hayka-kultura.com/larsen.html. Acesso em: 08 abr. 2025.

AMBRÓSIO, A.; ALMEIDA, L.; MACEDO, J.; SANTOS, A.; FRANCO, A. Programação de computadores: compreender as dificuldades de aprendizagem dos alunos. Revista Galego-Portuguesa de Psicoloxía e Educación, v. 19, n. 1, p. 185-197, dez. 2011.

BENNEDSEN, J.; CASPERSEN, M. E. Failure Rates in Introductory Programming: 12 Years Later. ACM SIGCSE Bulletin, v. 39, n. 2, p. 111–115, 2007.

DETERDING, S.; DIXON, D.; KHALED, R.; NACKE, L. From Game Design Elements to Gamefulness: Defining “Gamification”. In: MINDTREK CONFERENCE, 2011, Tampere. Proceedings... Tampere: ACM, 2011.

FERREIRA, C.; GONZAGA, F.; SANTOS, R. Um Estudo sobre a Aprendizagem de Lógica de Programação Utilizando Programação por Demonstração. In: WORKSHOP SOBRE EDUCAÇÃO EM COMPUTAÇÃO – CONGRESSO DA SOCIEDADE BRASILEIRA DE COMPUTAÇÃO, 2010. Anais... SBC, 2010.

FLOYD, D. Video Games and Learning. [YouTube]. 2008. Disponível em: https://www.youtube.com/watch?v=rN0qRKjfX3s. Acesso em: 08 abr. 2025.

GEE, J. P. What Video Games Have to Teach Us About Learning and Literacy? New York: Palgrave Macmillan, 2003.

HAMARI, J.; KOIVISTO, J.; SARSA, H. Does Gamification Work? – A Literature Review of Empirical Studies on Gamification. In: HAWAII INTERNATIONAL CONFERENCE ON SYSTEM SCIENCES (HICSS), 47., 2014. Proceedings... IEEE, 2014.

MARTINS, M. W. P.; MARIN, P. M.; ALVES, B. L. Desafios na aprendizagem de lógica de programação entre estudantes de tecnologia: uma revisão sistemática da literatura. Revista EDaPECI, v. 24, n. 3, p. 12–24, dez. 2024.

PONTES, H. P. Desenvolvimento de jogos no processo de aprendizado em algoritmos e pro-gramação de computadores. SBGames, n. De-cember, p. 220–228, 2013.

NICHOLSON, S. A Recipe for Meaningful Gamification. In: LANDERS, R. N. (org.). Gamification in Education and Business. Berlin: Springer, 2015. p. 1–20. https://doi.org/10.1007/978-3-319-10208-5_1

PAPERT, S. Mindstorms – Children, Computers and Powerful Ideas. New York: Basic Books, 1980.

PORTNOW, J. Tangential Learning – How Games Can Teach Us While We Play. [YouTube]. 2012. Disponível em: https://youtube.com/watch?v=rlQrTHrwyxQ. Acesso em: 08 abr. 2025.

SILVA, B. Aprendizagem tangencial no processo de ensino e aprendizagem de conceitos científicos: um estudo de caso. RENOTE – Revista Novas Tecnologias na Educação, v. 14, n. 2, 2016. Disponível em: https://seer.ufrgs.br/index.php/renote/article/view/70678. Acesso em: 14 abr. 2025.

SMITH, D. C. Building personal tools by programming. Communications of the ACM, v. 43, n. 8, p. 92–95, ago. 2000.

G. Petri, C. Gresse von Wangenheim, A.F. Borgatto A.F. MEEGA+, Systematic Model to Evaluate Educational Games. In: Lee N. (eds) Encyclopedia of Computer Graphics and Games. Springer, 2018.

GLOSSÁRIO

Programming by Demonstration (PBD)	Técnica que permite ao usuário ensinar o sistema a executar tarefas através de demonstrações, ao invés de escrever código diretamente (Ferreira, Gonzaga e Santos, 2010).
Aprendizagem Tangencial	Método de ensino no qual o aprendizado ocorre de forma indireta e contextualizada, muitas vezes por meio da exposição a conceitos durante atividades lúdicas (Silva, 2016).
Gamificação	Aplicação de elementos de design de jogos em contextos não lúdicos, com o objetivo de aumentar o engajamento e a motivação dos usuários (Deterding et al., 2011).
Jogo Sério	Jogo digital cujo propósito principal vai além do entretenimento, sendo utilizado para educação, treinamento ou conscientização (Abt, 1970).
LMS (Learning Management System - Sistema de Gestão de Aprendizagem)	Plataforma para gestão de conteúdos educacionais, permitindo acompanhamento do desempenho dos alunos.
NPC (Non-Playable Character - Personagem Não Jogável)	Personagem não controlado pelo jogador, usado para narrativa ou interação em jogos.
Sprint	Ciclo curto e fixo de tempo em que uma equipe realiza um conjunto de tarefas em metodologias ágeis.
Checkpoint	Ponto de salvamento em jogos, permitindo que o progresso seja retomado posteriormente.
Engine (motor de jogo)	Software utilizado para desenvolvimento de jogos, como a Godot Engine citada no projeto.
Godot Engine	Ferramenta de código aberto para desenvolvimento de jogos 2D e 3D.
WebGL	Biblioteca gráfica baseada em JavaScript que permite renderização 2D e 3D em navegadores.
JSON (JavaScript Object Notation - Notação de Objetos JavaScript)	Formato de dados baseado em texto, usado para armazenar e trocar dados de forma simples e eficiente.